

Anchored Bidirectional Half-State Evaluation for Exact Group Steiner Tree Search

Anonymous Authors
Anonymous Institution
Anonymous City, Anonymous

1 Preliminaries and Reading Guide

Let $G = (V, E, w)$ be an undirected graph with nonnegative edge weights, where $n = |V|$ and $m = |E|$. A query $\mathcal{T} = \{T_1, \dots, T_g\}$ contains g vertex groups, which may overlap. A group Steiner tree (GST) is a connected subgraph that intersects every T_i . Its weight is the sum of its edges, and the exact GST problem asks for a minimum-weight GST. Zero-weight edges are allowed. A one-group query has value zero; a query is infeasible if no connected component intersects all groups.

Write $[g] = \{1, \dots, g\}$ and $d_i(v) = \text{dist}(v, T_i)$. Let $d = (d_1, \dots, d_g)$ denote the complete collection of group-distance functions, obtained by g multi-source shortest-path searches. For a vertex function $f : V \rightarrow \mathbb{R}_{\geq 0} \cup \{\infty\}$, define its *graph closure*

$$C(f)(v) = \min_{u \in V} \{f(u) + \text{dist}(u, v)\}. \quad (1)$$

A closure moves the root of every finite seed through the graph and is evaluated by one multi-source shortest-path search. Classical exact GST dynamic programming alternates two operations: it joins disjoint group subsets at a common root, then applies (1) [2, 6]. Its 2^g rooted subset rows and 3^g subset joins motivate a different organization of the same exact recurrence.

Reading guide. Section 2 first gives the global state organization, a running example, and the complete algorithm. Section 3 then explains how anchor-free rows are generated and safely pruned. Section 4 follows one anchored derivation from its low forward rows, through the forward/backward boundary, to its high terminal. Section 5 proves that this reorganization preserves every optimum derivation and gives its complexity. Thus a reader may understand the full data flow from Section 2 before entering individual optimizations.

2 Algorithmic Roadmap

2.1 Anchor and state roles

Choose one group T_a as a permanent *anchor*, and let $K = [g] \setminus \{a\}$ and $k = |K| = g - 1$. The anchor gives every partial solution a unique distinguished side. Let

$$h = \lfloor g/2 \rfloor, \quad a^{\max} = h - 1, \quad c = \lfloor a^{\max}/2 \rfloor.$$

The value h is the largest ordinary block required by a balanced three-part decomposition. Because the anchored part already contains T_a , it needs at most $a^{\max}$ additional groups. The cut c divides this anchored range by mask depth, not by graph or query statistics.

Table 1 defines every state family and, equally importantly, its role. A subscript is always a mathematical index such as a group mask or vertex. Textual qualifiers appear as superscripts.

The algorithm computes D_S for $1 \leq |S| \leq h$. A complete forward anchored DP would compute A_S up to $|S| = a^{\max}$. We instead compute only its low rows $|S| \leq c$, and evaluate the remaining

Table 1: State and parameter roles.

Symbol	Meaning and role
B	Weight of the best feasible GST found so far; it is an upper bound on OPT.
$D_S(v)$	Minimum tree rooted at v that covers the non-anchor mask $S \subseteq K$.
$\text{Br}_S(v)$	An exact $D_S(v)$ entry exported as one indivisible branch to later joins.
$A_S(v)$	Minimum tree rooted at v that covers T_a and S ; only $ S \leq c$ is stored.
$G_T(v)$	Cost of finishing high anchored mask T at v with at most two ordinary trees.
$F_T(v)$	High-mask suffix cost after the incoming graph closure has already been paid.
$H_T(v)$	$C(F_T)(v)$; the suffix pulled back to the root before that closure.

dependency DAG backward as H_T for $c < |T| \leq a^{\max}$. The two directions meet at the unique first edge that crosses from a low mask to a high mask. This is one exact dynamic program with a changed evaluation order, rather than two competing solvers.

2.2 Running example

Consider $g = 8$ and suppose preprocessing selects T_8 as the anchor, so $K = \{1, \dots, 7\}$. Then $h = 4$, $a^{\max} = 3$, and $c = 1$. Hence we store only $A_\emptyset$ and singleton A rows, while pair and triple anchored masks are evaluated backward.

At a root x , consider the forward derivation

$$A_{\{1\}} \xrightarrow{\text{Br}_{\{2\}}} A_{\{1,2\}} \xrightarrow{\text{Br}_{\{4\}}} A_{\{1,2,4\}},$$

followed by ordinary blocks $D_{\{3,5\}}$ and $D_{\{6,7\}}$. Suppose the five displayed values at x are 6, 2, 3, 4, 5, respectively, and the displayed closures attain their minima at x . Full forward evaluation obtains $(6 + 2 + 3) + (4 + 5) = 20$.

Our evaluation reaches the same value without storing either high A row. First, the ordinary pair $(\{3, 5\}, \{6, 7\})$ is transposed to its complement and emits $G_{\{1,2,4\}}(x) = 4 + 5 = 9$. Backward evaluation gives $H_{\{1,2,4\}}(x) = 9$ and then $H_{\{1,2\}}(x) = 3 + 9 = 12$. Finally, the first crossing edge is settled as

$$A_{\{1\}}(x) + \text{Br}_{\{2\}}(x) + H_{\{1,2\}}(x) = 6 + 2 + 12 = 20.$$

With nontrivial shortest paths, the closure identity in Section 4.2 moves the same path costs to the backward side. This example will be revisited when the terminal transpose is defined.

2.3 End-to-end procedure

Algorithm 1 uses named interfaces so that every line has one role. $\text{ForwardClose}(f, R, B)$ computes $C(f)$ using a lower bound for the unassigned labels R . $\text{PrefixClose}(f, X, B)$ instead bounds the already included labels X during a backward closure. Both use incumbent B .

Algorithm 1: Anchored bidirectional half-state evaluation.

Input : $G = (V, E, w)$ and groups $\mathcal{T} = \{T_1, \dots, T_g\}$
Output : The exact GST weight

```

1  $d \leftarrow \text{GroupDistances}(G, \mathcal{T})$ 
2  $r^* \leftarrow \text{BestCommonRoot}(d)$ 
3  $B \leftarrow \sum_{i \in [g]} d_i(r^*)$ 
4  $a \leftarrow \text{FarthestGroup}(r^*, d)$ 
5  $K \leftarrow [g] \setminus \{a\}$ 
6  $(h, a^{\max}, c) \leftarrow \text{HalfLimits}(g)$ 
7 Initialize the tour bound and the progressive cut bound
8 foreach  $i \in K$  do
9    $D_{\{i\}} \leftarrow d_i$ 
10   $\text{Br}_{\{i\}} \leftarrow D_{\{i\}}$ 
11  $B \leftarrow \text{Improve}(D, B)$ 
12 for  $s \leftarrow 2$  to  $h$  do
13   foreach  $S \subseteq K$  with  $|S| = s$  do
14      $M_S \leftarrow \text{OrdSeed}(S, D, \text{Br})$ 
15      $D_S \leftarrow \text{ForwardClose}(M_S, \{a\} \cup (K \setminus S), B)$ 
16      $\text{Br}_S \leftarrow \text{Publish}(D_S, M_S)$ 
17     Advance the cut bound using measured row work
18      $B \leftarrow \text{Improve}(D, B)$ 
19  $A_0 \leftarrow d_a$ 
20  $B \leftarrow \text{CompleteLow}(\emptyset, A_0, D, B)$ 
21 for  $s \leftarrow 1$  to  $c$  do
22   foreach  $S \subseteq K$  with  $|S| = s$  do
23      $M_S^A \leftarrow \min_{\emptyset \neq Q \subseteq S} (A_{S \setminus Q} + \text{Br}_Q)$ 
24      $A_S \leftarrow \text{ForwardClose}(M_S^A, K \setminus S, B)$ 
25      $B \leftarrow \text{CompleteLow}(S, A_S, D, B)$ 
26  $G \leftarrow \text{HighTerminals}(D, K, a, c, a^{\max}, B)$ 
27 for  $s \leftarrow a^{\max}$  downto  $c + 1$  do
28   foreach  $T \subseteq K$  with  $|T| = s$  do
29      $F_T \leftarrow \text{OutsideSeed}(T, G, \text{Br}, H)$ 
30      $H_T \leftarrow \text{PrefixClose}(F_T, \{a\} \cup T, B)$ 
31      $B \leftarrow \text{CrossBoundary}(T, A, \text{Br}, H_T, B)$ 
32 return  $B$ 
```

Named interfaces. Section 3.1 defines OrdSeed and Publish. Section 4.1 defines CompleteLow. Section 4.2 defines OutsideSeed and CrossBoundary. Section 3.3 gives the feasible solutions constructed by Improve. HighTerminals is Algorithm 2.

Reading Algorithm 1. Line 1 materializes the group-distance collection d . Lines 2–4 use it to choose the best common root, build the first feasible incumbent, and select the anchor. Lines 5–7 derive the mask limits and initialize the two static bound families. Lines 8–10 create the singleton ordinary rows; line 11 immediately lets the feasible constructions use them.

Lines 12–18 build every remaining ordinary row in increasing mask size. A row is joined, forward-closed, exported through the branch interface, and then used to purchase stronger bounds or feasible solutions. Lines 19–25 compute the low anchored rows and settle every completion ending before the cut. Line 26 transposes ordinary completions into all high targets at once. Lines 27–31 traverse the high dependency DAG in decreasing mask size and settle each first crossing from a stored low row. After every high row and crossing has been processed, line 32 returns the exact incumbent.

3 Ordinary Rows and Safe Bounds

3.1 Ordinary rows and the branch interface

For a singleton, $D_{\{i\}} = d_i$. For $|S| > 1$, let $p(S)$ be the least group index in S . The *ordinary seed* keeps $p(S)$ on the accumulated side and attaches a nonempty branch Q from the other side:

$$M_S = \text{OrdSeed}(S, D, \text{Br}) = \min_{\substack{\emptyset \neq Q \subseteq S \\ p(S) \notin Q}} \{D_{S \setminus Q} + \text{Br}_Q\}, \quad D_S = C(M_S). \quad (2)$$

All minima and additions of vertex functions are pointwise. After closure, $D_S(v)$ is published as an indivisible branch exactly when it was not already obtainable by a same-root split:

$$\text{Br}_S(v) = \begin{cases} D_S(v), & |S| = 1, \\ D_S(v), & |S| > 1 \text{ and } D_S(v) < M_S(v), \\ \infty, & \text{otherwise.} \end{cases} \quad (3)$$

Thus Publish(D_S, M_S) returns the finite entries selected by (3). An equality is expanded again at that root instead of being exported as another equivalent branch.

LEMMA 1 (BRANCH-BASIS COMPLETENESS). *Equation (2) computes the same $D_S(v)$ values as the unrestricted rooted subset recurrence.*

PROOF. Induct on $|S|$. Every published entry is an exact D value, so the restricted seed cannot be smaller. Conversely, consider an optimum unrestricted split $(S \setminus Q, Q)$ at v . If $\text{Br}_Q(v)$ is finite, the split occurs in (2). Otherwise $D_Q(v) = M_Q(v)$, and $Q = P \dot{\cup} R$ for some $D_P(v) + \text{Br}_R(v) = D_Q(v)$. Move P to the accumulated side. Subadditivity at the common root gives $D_{(S \setminus Q) \cup P}(v) + \text{Br}_R(v) \leq D_{S \setminus Q}(v) + D_Q(v)$. This replacement strictly shrinks the unpublished side, so iteration reaches a published branch without increasing cost. Both seed functions, and hence their closures, coincide. $\square$

Finite row entries are stored in vertex order. For each row, exact byte counts choose among a sparse vector, a dense array, and a ranked bitmap. Same-root joins use ordered merge, binary lookup, or bitmap intersection according to exact probe counts. These layouts implement the same recurrence and require neither hashing nor a learned density threshold.

3.2 Admissible completion bounds

Suppose a partial derivation has paid cost x , is rooted at v , and has unassigned labels R . It survives exactly when $x + L(v, R) \leq B$, where

$$L(v, R) = \max\{L^{\text{far}}(v, R), L^{\text{tour}}(v, R), L^{\text{cut}}(v, R)\}. \quad (4)$$

For overlapping groups, “unassigned” refers to a fixed representative hit for each group in an optimum, not to physical absence from the current subgraph. This certificate ensures that an accidental hit cannot invalidate the lower-bound argument.

All three terms are zero when $R = \emptyset$. For a singleton $R = \{i\}$, set $\text{HP}(R, i, i) = 0$, so the tour expression below reduces to $d_i(v)$.

The first term is the mandatory farthest connection, $L^{\text{far}}(v, R) = \max_{i \in R} d_i(v)$.

For the tour bound, let $\delta(i, j)$ denote the distance between groups T_i and T_j . Let $\text{HP}(R, i, j)$ be the shortest Hamiltonian path through labels R under δ , with endpoints i, j . A subset DP computes HP,

and doubling any completion tree proves

$$L^{\text{tour}}(v, R) = \frac{1}{2} \min_{i, j \in R} \{d_i(v) + \text{HP}(R, i, j) + d_j(v)\}.$$

The third term shares edge capacity across groups. Temporarily view each undirected edge in both accounting directions. Group potentials π_i are zero on T_i and satisfy, for every accounting arc (u, v) ,

$$\sum_i [\pi_i(u) - \pi_i(v)]_+ \leq w(u, v). \quad (5)$$

Orienting any completion tree away from v charges every tree edge in only one direction, proving $L^{\text{cut}}(v, R) = \sum_{i \in R} \pi_i(v)$. Potentials are constructed by dual ascent on residual arc capacities [7]. Initially $r(u, v) = w(u, v)$. When group i is processed, let q_i be its distance-to- T_i function under r , set

$$\begin{aligned} \pi_i(v) &= \min\{q_i(v), q_i(r^*)\}, \\ r(u, v) &\leftarrow r(u, v) - [\pi_i(u) - \pi_i(v)]_+. \end{aligned}$$

The truncation makes r^* the common cap, while residual subtraction preserves (5). To compute a later q_i , shortest-path propagation is seeded only by arcs whose residual was changed earlier. Unchanged arcs already satisfy the original distance triangle inequality, but remain available once propagation starts.

Constructing all potentials before a small search can cost more than it saves. Therefore every completed non-singleton D row deposits its exact seed probes, queue pops, and edge relaxations into a work bank. Each $C^{\text{dual}} = 2m + n$ units construct the next potential in a fixed farthest-first group order. Every prefix satisfies (5), while unbuilt groups have zero potential. If all potentials finish, unused residual capacity can be packed by scaling another family of truncated residual-distance potentials q_i . With active scales $0 \leq \lambda_i \leq 1$ and load $\ell(u, v) = \sum_i [q_i(u) - q_i(v)]_+$, the maximal safe step is

$$\Delta = \min \left\{ \min_i (1 - \lambda_i), \min_{\ell(u, v) > 0} \frac{r(u, v)}{\ell(u, v)} \right\}.$$

Each step fixes a group or saturates an arc, so packing terminates and preserves (5). Packing is attempted once only after later row work pays its explicit arc/group scan and at least one ordinary row remains to benefit; otherwise the residual structure is released. Work accounting changes only when a stronger valid bound becomes available; it never changes the state graph or stopping condition.

3.3 Feasible incumbent updates

The initial common-root solution has cost $U^{\text{root}}(r) = \sum_i d_i(r)$. We choose $r^* = \arg \min_r U^{\text{root}}(r)$, set $B = U^{\text{root}}(r^*)$, and choose a farthest group at r^* as the anchor, with group index breaking ties. Any anchor is correct; this deterministic choice favors a long path that later branches may share.

Improve maintains three additional feasible constructions. First, it restores a shortest r^* -to- T_a backbone, attaches every non-anchor group triple at its best common junction on that backbone, compresses degree-two paths, and runs exact subset DP on the resulting tree. Second, completed D rows supply block-to-backbone costs for a path-facility partition, where each assigned block is connected by a realized rooted tree and path. Third, once rows of size $\lceil k/3 \rceil$ exist, all balanced three-block partitions are joined with the anchor path; a bounded sequence of exact closures lets the anchor side

share a path with one selected block. Each candidate is a union of actual graph paths and rooted trees, hence can only lower B to another feasible value. Expensive tree evaluations are invoked after measured closure work covers their exact probe count.

4 Anchored Bidirectional Evaluation

4.1 Low inside rows and direct completions

The anchored base is $A_\emptyset = d_a$. A low row absorbs one ordinary branch and then closes:

$$A_S = C \left(\min_{\emptyset \neq Q \subseteq S} \{A_{S \setminus Q} + \text{Br}_Q\} \right), \quad |S| \leq c. \quad (6)$$

Only one operand contains the anchor, so every derivation has one anchored spine. $\text{CompleteLow}(S, A_S, D, B)$ partitions the remaining labels into two ordinary blocks and updates

$$B \leftarrow \min \left\{ B, \min_{\substack{L \cup R = K \setminus S \\ |L|, |R| \leq h}} \langle A_S, D_L + D_R \rangle \right\}, \quad (7)$$

where $D_\emptyset(v) = 0$ and $\langle f, q \rangle = \min_v \{f(v) + q(v)\}$. Every update is a connected union at one root.

4.2 High outside rows and the crossing boundary

Materializing A_S for $c < |S| \leq a^{\max}$ is unnecessary because graph closure is self-adjoint under the min-plus inner product:

$$\langle C(f), q \rangle = \min_{u, v} \{f(u) + \text{dist}(u, v) + q(v)\} = \langle f, C(q) \rangle. \quad (8)$$

For a high mask T , F_T denotes the suffix after the incoming closure, and $H_T = C(F_T)$ pulls that suffix back to the preceding merge root. Decreasing mask size makes every strict-superset dependency available:

$$F_T = \min \left\{ G_T, \min_{\substack{\emptyset \neq Q \subseteq K \setminus T \\ |T \cup Q| \leq a^{\max}}} \{\text{Br}_Q + H_{T \cup Q}\} \right\}, \quad (9)$$

$$H_T = C(F_T). \quad (10)$$

OutsideSeed is the right-hand side of (9). All strict supersets are necessary because a forward transition may absorb a multi-group branch and skip mask layers.

Every forward path that enters the high range has a unique first edge $S \rightarrow T = S \cup Q$ with $|S| \leq c < |T|$. CrossBoundary therefore scans these edges and updates

$$B \leftarrow \min\{B, \langle A_S, \text{Br}_Q + H_T \rangle\}. \quad (11)$$

In the running example, $T = \{1, 2\}$ and $Q = \{2\}$ identify exactly this first crossing; H_T already includes the later branch $\{4\}$ and the complement terminal.

Backward rows require a lower bound on the already built anchored prefix, rather than on the suffix. Let $P_T^{\text{anc}}(v)$ be the maximum of the far, tour, and cut bounds applied to $\{a\} \cup T$. Since $P_T^{\text{anc}}(v) \leq A_T(v)$, a suffix value q is discarded only if $q + P_T^{\text{anc}}(v) > B$. Each component is consistent along graph edges, so a discarded seed cannot become useful after closure. This prefix-aware closure is exactly $\text{PrefixClose}(F_T, \{a\} \cup T, B)$ in Algorithm 1.

4.3 Root-wise terminal transposition

For a high target T , its direct suffix is

$$G_T(v) = \min_{\substack{L \cup R = K \setminus T \\ |L|, |R| \leq h}} \{D_L(v) + D_R(v)\}. \quad (12)$$

Computing this expression separately for every T repeatedly reads the same ordinary rows. Algorithm 2 instead fixes a root, enumerates an ordinary pair once, and emits it to the unique complement $T = K \setminus (L \cup R)$. In the running example, the pair $(\{3, 5\}, \{6, 7\})$ emits directly to $T = \{1, 2, 4\}$.

Potentials provide a target-independent filter. Define $\pi_X(v) = \sum_{i \in X} \pi_i(v)$,

$$\rho^v(S) = D_S(v) - \pi_S(v), \quad \beta^v = B - \pi_{\{a\} \cup K}(v).$$

For every partition $L \dot{\cup} R \dot{\cup} T = K$,

$$D_L(v) + D_R(v) + \pi_{\{a\} \cup T}(v) = \rho^v(L) + \rho^v(R) + \pi_{\{a\} \cup K}(v). \quad (13)$$

Thus a pair with $\rho^v(L) + \rho^v(R) > \beta^v$ cannot improve B . Reduced values only filter and order events; G_T stores the true cost $D_L + D_R$.

Algorithm 2: Root-wise transposed high-terminal generation.

Input : Ordinary rows D , masks K , anchor a , cut c , limit $a^{\max}$, incumbent B

Output : High terminal rows G

```

1 foreach root  $v$  in streamed vertex blocks do
2    $E^v \leftarrow \{(\emptyset, 0)\}$ 
3   foreach ordinary row  $D_S$  containing  $v$  do
4     Insert  $(S, D_S(v))$  into  $E^v$ 
5    $\beta^v \leftarrow B - \pi_{\{a\} \cup K}(v)$ 
6    $z^{\text{sort}} \leftarrow \text{SortedProbeCount}(E^v, \beta^v)$ 
7    $z^{\text{sub}} \leftarrow \text{SubmaskProbeCount}(E^v, K)$ 
8    $\mathcal{P} \leftarrow \text{CheaperExactPlan}(z^{\text{sort}}, z^{\text{sub}})$ 
9   foreach unordered disjoint pair  $(L, R)$  returned by  $\mathcal{P}$  do
10    if  $\rho^v(L) + \rho^v(R) \leq \beta^v$  then
11       $T \leftarrow K \setminus (L \cup R)$ 
12      if  $c < |T| \leq a^{\max}$  then
13         $x \leftarrow D_L(v) + D_R(v)$ 
14        if  $x + p_T^{\text{anc}}(v) \leq B$  then
15           $G_T(v) \leftarrow \min\{G_T(v), x\}$ 
16 return  $G$ 

```

Reading Algorithm 2. Lines 1–4 gather, for one streamed root, all ordinary masks whose rows contain that root; the explicit empty row also represents one-block completions. Lines 5–8 compute the common reduced-cost budget and choose between two exact enumerations by their exact probe counts. The sorted plan scans value pairs under β^v and then tests disjointness. The submask plan enumerates $R \subseteq K \setminus L$ and looks up present rows. Lines 9–15 apply the common budget, map each surviving pair to one high complement, apply the full anchored-prefix bound, and retain its minimum true cost. Both plans enumerate the same disjoint pairs, and submask enumeration performs at most $\sum_{L \subseteq K} 2^{k-|L|} = 3^k$ probes. Line 16 returns the aggregated terminal rows.

5 Correctness and Complexity

LEMMA 2 (ANCHORED BALANCED DECOMPOSITION). *For any optimum GST and any anchor group, there is a root r and a partition $S \dot{\cup} L \dot{\cup} R = K$ such that $|S| \leq h - 1$, $|L|, |R| \leq h$, and the optimum*

is the union of one rooted component covering $T_a \cup S$ and two rooted components covering L and R .

PROOF. Remove cycles and redundant edges from an optimum, and fix one hit for each group. Give each selected vertex the number of labels selecting it. A weighted centroid r leaves components of weight at most h . Treat these component weights and labels at r as positive integer items of total g , each at most h . They fit in three bins of capacity h : while more than three items remain, merge the two smallest. If the two smallest exceeded h , the four smallest would sum to at least $2h + 2 > g$. Assign the bin containing the anchor hit to the anchored component. It has at most $h - 1$ non-anchor labels; the other bins give L and R . Group overlap and zero-weight edges do not change this counting argument. $\square$

LEMMA 3 (EXACT HIGH-RANGE REPLACEMENT). *All low completions in (7) together with all first-crossing values in (11) have the same minimum as materializing every forward A_T with $|T| \leq a^{\max}$.*

PROOF. A forward edge that attaches Q is the operator $\mathcal{T}_Q(f) = C(f + \text{Br}_Q)$. Equation (8) moves its closure to the suffix, so its reverse action is $q \mapsto \text{Br}_Q + C(q)$. Applying this identity in reverse topological order yields (9)–(10). Every high forward path has one first low-to-high edge and maps value-preservingly to one crossing term. Conversely, every outside term is a true terminal or the reversal of a true forward edge, so it reconstructs a forward derivation. $\square$

THEOREM 1 (EXACTNESS). *Algorithm 1 returns the optimum GST weight.*

PROOF. Lemma 1 preserves every ordinary row. Lemma 2 places an optimum in an $A_S + D_L + D_R$ completion inside the retained half-state envelope. Lemma 3 preserves every such completion across the low/high cut. Algorithm 2 is a bijective reordering from each disjoint ordinary pair to its complement target; (13) rejects only pairs already exceeding a feasible incumbent under an admissible prefix. The three terms of (4) lower-bound the fixed-representative completion certificate: mandatory distance, half a doubled completion tour, and edge capacity charged at most once by (5). Progressive potentials, residual packing, and prefix pruning preserve these inequalities. Therefore no improving optimum derivation is removed, while every update to B is a realizable connected solution. The returned value equals OPT. $\square$

Let

$$N^D = \sum_{i=1}^h \binom{k}{i}, \quad N^A = \sum_{i=0}^c \binom{k}{i},$$

$$N^H = \sum_{i=c+1}^{a^{\max}} \binom{k}{i}, \quad N = N^D + N^A + N^H.$$

If one graph closure costs $C^G = O(m + n \log n)$, all ordinary joins, anchored transitions, reverse edges, and transposed terminal probes contribute $O(n3^k)$. Group-tour DP costs $O(g^3 2^g)$; potentials and packing cost $O(gC^G + g^2 m)$; backbone and three-block feasible solutions cost $O((h+t)3^k)$ for compressed backbone-tree size $t \leq n$. The worst-case time is

$$O(NC^G + n3^k + gC^G + g^2 m + g^3 2^g + (h+t)3^k). \quad (14)$$

This is $O(\text{poly}(g)(n3^k + 2^k C^G))$, matching the exponential base of balanced MITM [4].

Let Z^{state} be the number of finite rooted values stored and E^{term} the number of aggregated high-terminal events. Space is

$$O(m + gn + g^2 2^g + t2^k + Z^{\text{state}} + E^{\text{term}}), \quad (15)$$

where $Z^{\text{state}} \leq nN$ and $E^{\text{term}} \leq nN^H$ in the dense worst case. Root streaming bounds transient transpose storage. The method combines balanced decomposition [4], exact GST recurrence [2, 5, 6], dual ascent [7], outside evaluation [3], and disjoint-subset enumeration [1] into one anchored inside/outside state organization.

References

- [1] Andreas Bjorklund, Thore Husfeldt, Petteri Kaski, and Mikko Koivisto. 2007. Fourier Meets Mobius: Fast Subset Convolution. In *Proceedings of the 39th Annual ACM Symposium on Theory of Computing*. ACM, New York, NY, USA, 67–74. <https://doi.org/10.1145/1250790.1250801>
- [2] Stuart E. Dreyfus and Robert A. Wagner. 1971. The Steiner Problem in Graphs. *Networks* 1, 3 (1971), 195–207. <https://doi.org/10.1002/net.3230010302>
- [3] Daniel Gildea. 2020. Efficient Outside Computation. *Computational Linguistics* 46, 4 (2020), 745–762. https://doi.org/10.1162/coli_a_00386
- [4] Yoichi Iwata and Takuto Shigemura. 2019. Separator-Based Pruned Dynamic Programming for Steiner Tree. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press, Palo Alto, CA, USA, 1520–1527. <https://doi.org/10.1609/aaai.v33i01.33011520>
- [5] Jiayu Li, Yahui Sun, Bojing Ma, Libang Chen, Mengxi Hu, Feng Zhang, and Rong-Hua Li. 2025. Fast Optimal Group Steiner Tree Search using GPUs. *Proceedings of the ACM on Management of Data* 3, 6, Article 327 (2025), 27 pages. <https://doi.org/10.1145/3769792>
- [6] Rong-Hua Li, Lu Qin, Jeffrey Xu Yu, and Rui Mao. 2016. Efficient and Progressive Group Steiner Tree Search. In *Proceedings of the 2016 International Conference on Management of Data*. ACM, New York, NY, USA, 91–106. <https://doi.org/10.1145/2882903.2915217>
- [7] Richard T. Wong. 1984. A Dual Ascent Approach for Steiner Tree Problems on a Directed Graph. *Mathematical Programming* 28 (1984), 271–287. <https://doi.org/10.1007/BF02612335>